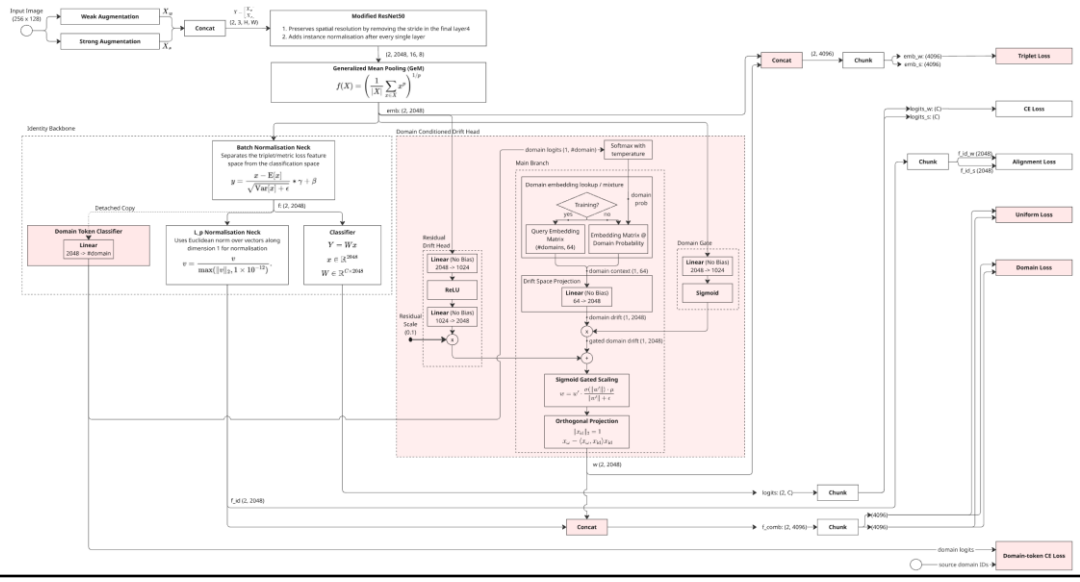
Last Week

1. [Administrative] Missing next week's meeting due to person holiday
2. [Model] Revisiting the proposed model from last week
3. [Loss] Changing from RandomIdentitySampler to RandomMultipleGallerySampler
4. [Model] Investigating the Instance Normalization Layers
5. [Loss] Investigating the Uniformity Loss + Identity-based Jaccard Similarity
6. [Model] Plan to migrate Finsler modifications to ReNorm

[Administrative] Next week's meeting cancelled

Reason: Doing a long weekend getaway before starting my Master's thesis. Meeting will resume on 03.04.

[Model] Architecture Proposal: Domain Conditioned Global Drift



Instead of predicting drift purely from the instance features, drift is now composed of: (a) a domain embedding looked up/mixed from source-domain labels, (b) projected and sigmoid-gated by instance features, (c) plus a small residual instance correction (scaled by 0.1)

The Foundation: The Identity Pathway

(Point to the top/left of the diagram)

- We maintain our modified ResNet50 (removing the final stride) and GeM pooling (*Radenović et al., 2018*) to preserve fine-grained spatial cues.
- The Batch Normalization neck and L_2 normalization cleanly project the identity into a view-invariant unit hypersphere.
    - **Role:** This branch acts as our rock. It focuses *only* on identity discrimination.

The Core Innovation: Domain Prediction & Soft Mixtures

Point to the "Domain Token Classifier" (Pink box, left).

- We branched off a *detached* copy of the identity feature into a linear Domain Classifier. This prevents domain-supervision gradients from corrupting the

core identity backbone (*Ganin & Lempitsky, 2015*).

Point to "Domain embedding lookup" (Center).

- During training, we know the source domains.
- But at evaluation on an *unseen* target domain, this classifier uses a temperature-scaled softmax to infer a soft probability mixture
    - (e.g., "This looks 70% CCTV, 30% Wearable").
- Instead of generating drift from scratch, we use those probabilities to mix learned domain embeddings.
- This enforces a crucial inductive bias: asymmetric drift is *shared* across identities based on view.

Adaptive Modulation: Gating and Residuals

Point to the "Domain Gate" and "Residual Drift Head".

- Not every image needs full domain drift.
    - A small neural gate dynamically scales the domain context so the model can gracefully fall back on identity for atypical samples (*Hu et al., 2018*).
- We still allow a tiny, per-instance residual correction (scaled down to 0.1) for lighting/pose noise.
    - This keeps the domain structure dominant while providing necessary flexibility.

Strict Disentanglement: Post-Processing

Point to the bottom of the Drift Head (Sigmoid Scaling & Orthogonal Projection).

- The sigmoid-gated scaling controls the total magnitude of the drift vector globally, allowing the model to shrink asymmetry to zero if it doesn't aid a specific batch.
- With the orthogonal projection, we mathematically force the drift vector to be orthogonal specifically to the identity vector
    - It ensures drift is strictly a nuisance/view factor.
    - It absolutely prevents the model from using drift as a shortcut to encode identity (*Kumar et al., 2018*).

Loss Integration & Conclusion

Point to the right side (Loss functions).

- BAU's Alignment loss strictly govern the identity slice to keep it augmentation invariant.
- Domain and Uniformity loss makes the embedding domain invariant.
- Our new Domain-Token CE loss trains the domain predictor.

Asymmetric Triplet: Finally, the full concatenated [Identity + Drift] vector is fed

into the Triplet Loss. Here, our asymmetric Finsler distance performs direction-aware hard-negative mining.

Conclusion: By explicitly factoring the embedding into a domain-invariant identity and a domain-conditioned directional drift,

- this architecture solves the instance-collapse problem.
- It allows us to apply structured, generalizable asymmetry to completely unseen camera domains.

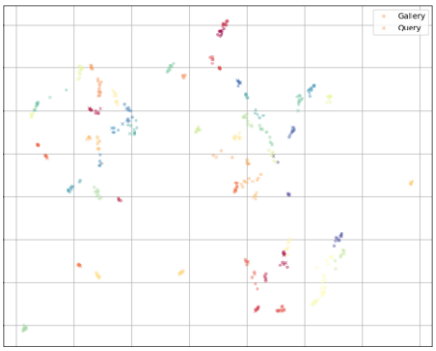
Negative samples should come from different domains

- Same identity from different domains using finsler distances
- Inject intuition from the equation
- Start simpler – add constrastive loss terms, then change the model

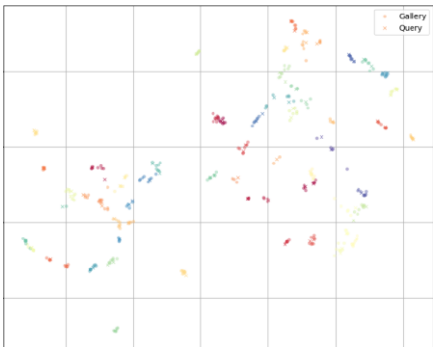
[Model] Architecture Proposal: Domain Conditioned Global Drift

	mAP		Rank 1	
Euclidean	42.21		42.5	
Drift Domain	43.89	+1.68	44.07	+1.57
Drift Instance	43.74	+1.53	44.21	+1.71

Instance Conditioned



Domain Conditioned



RandomIdentitySampler → RandomMultipleGallerySampler

Euclidean Batch-Hard Triplet Loss

$$\mathcal{L}_{BatchHard} = \frac{1}{B}\sum_{i=1}^{B} \max\left(0, \max_{p\in\mathcal{P}(i)} \|f(x_i) - f(x_p)\|_2 - \min_{n\in\mathcal{N}(i)} \|f(x_i) - f(x_n)\|_2 + \alpha\right)$$

Finsler Batch-Hard Triplet Loss

$$\mathcal{L}_{BatchHard} = \frac{1}{B}\sum_{i=1}^{B} \max\left(0, \max_{p\in\mathcal{P}(i)} \{d_f(f(x_i), f(x_p))\} - \min_{n\in\mathcal{N}(i)} \{d_f(f(x_i), f(x_n))\} + \alpha\right)$$

RandomIdentitySampler
Standard PxK sampling over identities and instances – does not use camera IDs, i.e., not guaranteed that the K instances per identity come from K different cameras.

RandomMultipleGallerySampler
Explicitly uses camera IDs: For each identity it picks one anchor, then fills the remaining K-1 with instances from other cameras.

- Looked into the triplet loss formulation -> it already is batch hard
- We had the idea of enforcing the positive to be from a different domain
    - But none of the identities appeared across different domains as the datasets were created separately from each other
    - Which means we can only enforce different camera id's between the x_i and x_p
- The set P(i) is filtered out using the labels and the maximum entry in the distance matrix is taken -> which means the sampler dictates the set
- Proceeded to look into the sampler
    - Found out that

So when an identity has at least K cameras, the K instances are guaranteed to be from K **different** cameras; otherwise it falls back to different instances (possibly same camera).

Rationale
- Intuition is asymmetry is due to camera/view geometry, not per-identity noise.
- To learn that, model must see cross-camera pairs in the batch.
- RandomIdentitySampler can give many same-camera pairs -> little or no

gradient for cross-view asymmetry.

RandomIdentitySampler → RandomMultipleGallerySampler

	mAP		Rank 1	
Euclidean + RandomIdentity (B)	42.21		42.5	
Euclidean + RandomMultiGallery	43.26	+1.05	43.0	+0.5
F-Instance + RandomIdentity	43.74	+1.53	44.21	+1.71
F-Instance + RandomMultiGallery	43.11	+0.9	42.29	-0.21
F-Domain + RandomIdentity	43.89	+1.68	44.07	+1.57
F-Domain+ RandomMultiGallery	44.00	+1.79	44.43	+1.93

Cross-camera sampler improves every reported metric for the domain-conditioned drift.

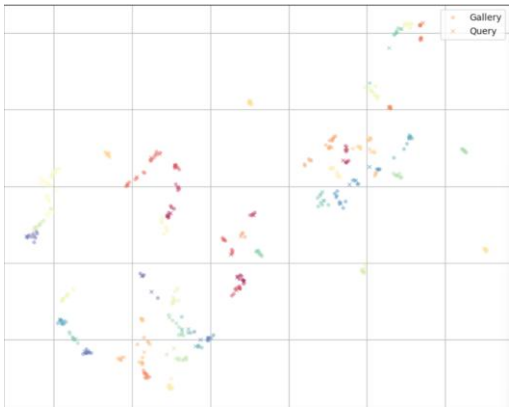
- Intra-identity variation largely due to camera-specific artifacts (lighting, background, viewpoint)
    - Identity-only sampling only gives “easy” same-camera positives
- Triplet and alignment loss get meaningful gradients for view-invariant alignment

Why this hurts the instance-conditioned drift vector

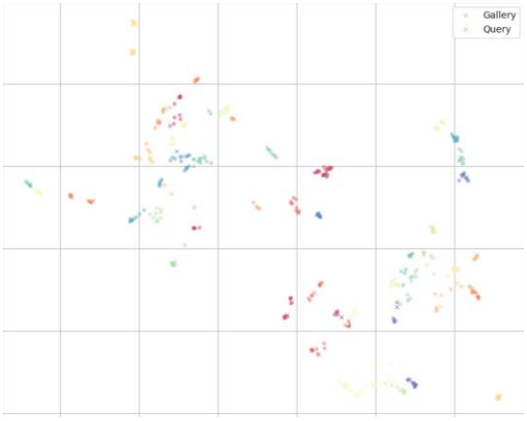
- On RMG, positives are visually far within the same ID.
- The asymmetric distance depends on instance-specific drift for both anchor and positive.
- We’re asking a per-instance channel to reconcile large cross-view appearance gaps together with strong ID alignment / CE / triplet and BAU’s uniform / domain pressures.
- Creates a gradient conflict: triplet + alignment want one consistent notion of “same person” across cameras; F-Instance invites pair- and sample-dependent metric tweaks.

RandomIdentitySampler → RandomMultipleGallerySampler

F-Domain+ RandomMultiGallery



Euclidean Baseline BAU



Do we need the Instance Normalization Layers?

```python
class resnet50(nn.Module):
    def __init__(self, num_classes=0, pretrained=True):
        super(resnet50, self).__init__()
        self.num_classes = num_classes

        # resnet50 backbone
        resnet = torchvision.models.resnet50(pretrained=pretrained)

        resnet.layer4[0].conv2.stride = (1, 1)
        resnet.layer4[0].downsample[0].stride = (1, 1)

        self.base = nn.Sequential(
            resnet.conv1, resnet.bn1, resnet.relu, resnet.maxpool,
            resnet.layer1, nn.InstanceNorm2d(256),
            resnet.layer2, nn.InstanceNorm2d(512),
            resnet.layer3, nn.InstanceNorm2d(1024),
            resnet.layer4, GeneralizedMeanPoolingP(output_size=(1, 1))
        )

        # pooling
        self.pool = GeneralizedMeanPoolingP(output_size=(1, 1))

        # bnneck and classifier
        self.bn_neck = nn.BatchNorm1d(2048)
        init.constant_(self.bn_neck.weight, 1)
        init.constant_(self.bn_neck.bias, 0)
        self.bn_neck.bias.requires_grad_(False)
        if self.num_classes > 0:
            self.classifier = nn.Linear(2048, self.num_classes, bias=False)
```

Two at Once: Enhancing Learning and Generalization Capacities via IBN-Net

Xingang Pan[1], Ping Luo[1], Jianping Shi[2], and Xiaoou Tang[1]

[1] CUHK-SenseTime Joint Lab, The Chinese University of Hong Kong
{px117,pluo,xtang}@ie.cuhk.edu.hk
[2] SenseTime Group Limited
shijianping@sensetime.com

JIA ET AL.: GENERALIZING PERSON RE-ID IN PRACTICE 1

Frustratingly Easy Person Re-Identification: Generalizing Person Re-ID in Practice

Jieru Jia[1]
12112059@bjtu.edu.cn
Qiuqi Ruan[1]
qqruan@bjtu.edu.cn
Timothy M. Hospedales[2]
thospedales@ed.ac.uk

[1] Institute of Information Science
Beijing Jiaotong University
Beijing, China
[2] Institute of Perception, Action and Behaviour
University of Edinburgh,
Edinburgh, UK

This is a ResNet-IN backbone, which is a result of InstanceNorm being known for style normalization.

- Paper 1 (2018): Introduces IBN-a / IBN-b; ReID is part of the story. IN mixed into ResNet for cross-domain
- Paper 2 (2019): Explicit “IN only in shallow ResNet stages, not in the deepest stage” for domain-generalizable ReI

ReNorm’s core claim is the usual BN–IN tradeoff pushed one step further:

- IN strips domain-specific statistics, which helps domain invariance but can remove identity-relevant signal
- they address that with Remix Normalization + Emulation Normalization and Domain Frozen, not by “more IN.”

IN sits before layer 4, so it shapes the pooled descriptor emb that feeds both the BN neck and drift head

- Preconditions what statistics reach emb
- Removing it is an ablation test of whether **explicit drift replaces implicit static IN**.

Uniformity Loss Requires Symmetric Distances + Identity-Based Jaccard

$$\mathcal{L}_{\text{uniform}} = \log\left(\frac{1}{|\mathcal{I}_{\text{data}}|}\sum_{(i,j)\in\mathcal{I}_{\text{data}}} e^{-2\|\mathbf{f}_i-\mathbf{f}_j\|_2^2}\right) + \log\left(\frac{1}{|\mathcal{I}_{\text{data}}|}\sum_{(i,j)\in\mathcal{I}_{\text{data}}} e^{-2\|\tilde{\mathbf{f}}_i-\tilde{\mathbf{f}}_j\|_2^2}\right), \quad (6)$$

where $\mathcal{I}_{\text{data}} = \{(i,j) \mid i \neq j\}$ is the index set of all distinct pairs within a mini-batch. By

Uniformity assumes a symmetric distance to push embeddings towards uniform coverage of the hypersphere

- Applying it with asymmetric distance might create an ill-defined optimization objective
- Asymmetric points not constrained to the sphere might *invalidate* the usual "uniform on the hypersphere minimizer" story from the original BAU narrative

Uniformity loss should operate on identity-only features with Euclidean distance.

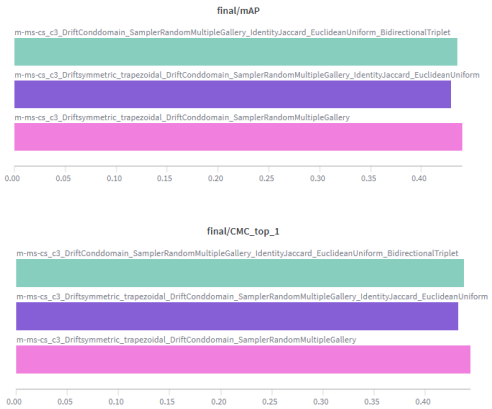
relationship with nearest neighbors [8, 9, 32, 84, 100], we compute the weight as the Jaccard similarity of k-reciprocal nearest neighbors between the augmented sample and the original sample, defined by:

$$w_{ij} = \frac{|\mathcal{R}_k(\tilde{\mathbf{f}}_i) \cap \mathcal{R}_k(\mathbf{f}_j)|}{|\mathcal{R}_k(\tilde{\mathbf{f}}_i) \cup \mathcal{R}_k(\mathbf{f}_j)|} \in [0,1], \quad (4)$$

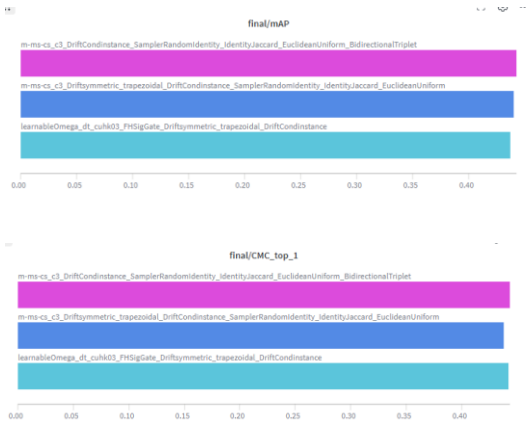
where $\mathcal{R}_k(\mathbf{f}_i)$ is the set of indices for k-reciprocal nearest neighbors within a mini-batch of the feature $\mathbf{f}_i$. Intuitively, a low weight implies that the augmented feature $\tilde{\mathbf{f}}_i$ and the original feature $\mathbf{f}_j$

Uniformity Loss Requires Symmetric Distances + Identity-Based Jaccard

Domain Conditioned



Instance Conditioned



General Critiques on Current Architecture Proposal

1. Adding a 2048-dimensional drift vector doubles the embedding dimensionality
   - *Any marginal improvement could be an artifact of increased capacity rather than asymmetric geometry*
   - To be monitored in ablation
2. Domain embedding only has 64 dimensions, projected to 2048
   - *Rank-64 bottleneck, i.e., domain-conditioned drift can only span a 64 dimensional subspace. Sufficient to capture meaningful asymmetry between domain shifts?*
   - With $S = 3$ source domains, the domain embedding matrix $E \in \mathbb{R}^{3\times \mathrm{d}}$ is projected through $W \in \mathbb{R}^{\mathrm{d}\times 2048}$ to produce the domain drift basis $M = EW \in \mathbb{R}^{3\times 2048}$. The rank of M is bounded by $\min(3, d)$. For any $d \geq 3$, the domain-prior subspace is at most rank 3.
3. The drift vector is conditioned on the domain-id of the identity, while no cross-domain identities exist
   - Sampler can only ensure that positives come from different cameras; metric learning signal from triplet loss does not complement modified architecture
   - Modify the conditioning to be based on camera-id and not domain-id

The sigmoid gate $\sigma(\text{Linear}(x)) \in \mathbb{R}^{2048}$ applies element-wise modulation per instance, but this does not increase the rank of the domain prior itself — it only rescales the fixed basis vectors per-sample. Sweeping $d \in \{32, 64, 128\}$ therefore changes the number of dead parameters and the conditioning of the projection matrix, but not the representational capacity of the domain prior.

This Week

1. [Model] Modified Triplet Loss Terms
2. [Model] Finslerized ReNorm

Next Week

1. [Sweep] Softmax temperature and domain token loss weight
2. [Ablation] Instance Normalization Layers
3. [Sweep] Omega regularizer scalar weight
4. [Sweep] Scalar multiplier for residual correction term

[Model] Modified Triplet Loss Terms

Motivation: Clean signal separation between identity-related and domain-related loss terms

Idea 1: Split Triplet Loss into Identity-Euclidean + Domain Finsler

$$\mathbf{x}_i = (\mathbf{x}_i^{\text{id}}\ \mathbf{x}_i^{\omega})^{\intercal}$$

1a. Euclidean Identity Triplet: Euclidean distance, RandomIdentitySampler, batch-hard mining on pid

$$\mathcal{L}_{tri}^{id} = \frac{1}{B}\sum_{i}^{N}\left[\max_{p:y_p=y_i} d_E(\mathbf{x}_i^{\text{id}}, \mathbf{x}_p^{\text{id}}) - \min_{n:y_n\neq y_i} d_E(\mathbf{x}_i^{\text{id}}, \mathbf{x}_n^{\text{id}}) + m_{id}\right]_+$$

1b. Finsler Domain Triplet: Finsler distance, RandomDomainSampler, batch-hard mining on domain/camera

$$\mathcal{L}_{tri}^{dom} = \frac{1}{B}\sum_{i}^{B}\left[\max_{p:d_p=d_i} d_F(\mathbf{x}_i, \mathbf{x}_j) - \min_{n:d_n\neq d_i} d_F(\mathbf{x}_i, \mathbf{x}_n) + m_{dom}\right]_+$$

$$d_F(\mathbf{x}, \mathbf{y}) = \|\mathbf{x}_{\text{id}} - \mathbf{y}_{\text{id}}\|_2 + \langle \frac{\mathbf{x}_\omega + \mathbf{y}_\omega}{2}, \mathbf{y}_{id} - \mathbf{x}_{id}\rangle$$

d_i Is the domain/camera label

- Facilitates feature disentanglement and isolates the contributions of asymmetric (Finsler) geometry to domain modelling

Rationale
- Identity invariance stays Euclidean (same-person features close regardless of domain)
- Domain structure is modeled via Finsler (same-domain features clustered in asymmetric space)

Consideration
- We can still fine-tune the margin for Finsler Domain Triplet

Critical analysis
- Domain triplet pulls same-domain embeddings closer and pushes different-domain embeddings apart
- Existing domain loss does the exact opposite; it repels nearest same-domain memory bank neighbors
- If we proceed with domain triplet we will be clustering by domain or camera, which is the failure mode of domain-generalizable person re-identification.
    - Re-introduce camera-style shortcuts
    - Loss learns to cluster arbitrary identities by domain

If we want to learn what is different between domains (drift vector models domain artifacts)
- Positive should be cross-domain same-identity
- Negative should be same-domain same-identity

[Model] Modified Triplet Loss Terms

Motivation: Clean signal separation between identity-related and domain-related loss terms

Idea 1: Split Triplet Loss into Identity-Euclidean + Domain Finsler

$$\mathbf{x}_i = (\mathbf{x}_i^{\text{id}}\ \mathbf{x}_i^{\omega})^\intercal$$

1a. Euclidean Identity Triplet: Euclidean distance, RandomIdentitySampler, batch-hard mining on pid

$$\mathcal{L}_{tri}^{id} = \frac{1}{B}\sum_{i}^{N}\left[\max_{p:y_p=y_i} d_E(\mathbf{x}_i^{\text{id}}, \mathbf{x}_p^{\text{id}}) - \min_{n:y_n\neq y_i} d_E(\mathbf{x}_i^{\text{id}}, \mathbf{x}_n^{\text{id}}) + m_{id}\right]_+$$

1b. (Refined) Cross-domain constrastive loss: Finsler distance, RandomDomainSampler

$$\mathcal{L}_{contrast}^{dom} = \frac{1}{|\mathcal{C}|}\sum_{(i,j)\in\mathcal{C}} d_F(\mathbf{x}_i, \mathbf{x}_j)^2 \qquad \mathcal{C} = \{(i,j) : y_i = y_j, d_i \neq d_j\}$$

$$d_F(\mathbf{x}, \mathbf{y}) = \|\mathbf{x}_{\text{id}} - \mathbf{y}_{\text{id}}\|_2 + \langle \frac{\mathbf{x}_\omega + \mathbf{y}_\omega}{2}, \mathbf{y}_{id} - \mathbf{x}_{id}\rangle$$

d_i Is the domain/camera label

"We want to compare images from different domains, not necessarily same identity"

- Loss can be formulated as a cross-domain constrastive on identity features via Finsler distance
- Encourages the Finsler distance to shrink cross-domain same-identity pairs
    - Asymmetric term must compensate for the domain drift

[Model] Modified Triplet Loss Terms

Motivation: Clean signal separation between identity-related and domain-related loss terms

Idea 1: Split Triplet Loss into Identity-Euclidean + Domain Finsler

$$\mathbf{x}_i = (\mathbf{x}_i^{\mathrm{id}}\ \mathbf{x}_i^{\omega})^{\intercal}$$

1a. Euclidean Identity Triplet: Euclidean distance, RandomIdentitySampler, batch-hard mining on pid

$$\mathcal{L}_{tri}^{id} = \frac{1}{B}\sum_{i}^{N}\left[\max_{p:y_p=y_i} d_E(\mathbf{x}_i^{\mathrm{id}}, \mathbf{x}_p^{\mathrm{id}}) - \min_{n:y_n\neq y_i} d_E(\mathbf{x}_i^{\mathrm{id}}, \mathbf{x}_n^{\mathrm{id}}) + m_{id}\right]_+$$

1c. Euclidean L2 Loss on Drift Vector: Euclidean distance, RandomDomainSampler

$$\mathcal{L}_{\text{drift-xcontrast}} = \frac{1}{|\mathcal{C}|}\sum_{(i,j)\in\mathcal{C}} \|\boldsymbol{\omega}_i - \boldsymbol{\omega}_j\|_2^2 \quad \mathcal{C} = \{(i,j) : y_i = y_j,\ c_i \neq c_j\}$$

[Model] Modified Triplet Loss Terms -- Results

Short Name	Description	Drift Conditioning	mAP	Rank-1
Baseline 1a only	Euclidean identity triplet only (Idea 1a); no cross-domain contrastive; no domain triplet; no drift-only L2	instance	43.7	43.4
		domain	44.1	45
1a + 1b (refined)	Refined 1b: Finsler distance on same-PID pairs with different global camera id; weight 0.1	instance	42.2	42.2
		domain	42.8	44.1
1a + 1b + no domain loss	Batch-hard triplet on camera labels using full Finsler embedding; domain_triplet_weight=1.0; BAU memory domain loss disabled (--no-domain)	instance	24.6	30.4
		domain	24.6	30.4
1a + 1c	Mean squared L2 on drift vectors for same PID different camera; weight 0.1	instance	43	43.1
		domain	42.8	41.9
1a + 1b	Same domain triplet as 3rd but BAU memory domain loss enabled (no --no-domain)	instance	35.5	36.4
		domain	36.2	36.1

- refined 1b should pull **same-ID, different-camera** pairs together under (d_F) without contradicting DG domain invariance as strongly as “same-domain attraction” on arbitrary IDs
- **domain triplet** implements batch-hard mining on **camera** as a “nuisance label” — anticipated to **conflict** with BAU’s memory **domain loss** (attraction vs repulsion on domain/camera structure)

Previously

	M+MS+CS → C3		M+CS+C3 → MS		MS+CS+C3 → M		Average	
	mAP	Rank-1	mAP	Rank-1	mAP	Rank-1	mAP	Rank-1
CLIP-FGDI	**44.4**	44.6	**31.1**	**59.4**	**79.4**	**91.3**	**51.6**	**65.1**
ReNorm	43.6	44.7	25.6	55.6	72.7	89.1	47.3	63.1
ReNorm*	42.4	41.5	24.2	53.3	72.6	88.9	46.4	61.2
BAU	42.8	43.9	24.3	50.9	77.1	90.4	48.1	61.7
BAU*	42.21	42.5	23.28	49.70	76.61	89.61	47.37	60.60
Finsler BAU*	43.95	43.93	22.22	48.04	76.47	89.91	47.55	60.63

[Model] Finslerized ReNorm

short_name	drift_conditioning	mAP	Rank1	eval_epoch
Baseline euclidean	Final Epoch	42.36	41.5	119
	Peak MAP	43.05	42.71	79
unified_finsler	instance			
	domain	42.21	41.79	69
1a + 1b	instance			
	domain	35.55	34.29	69
1a + 1b (refined)	instance	14.76	13.29	59
	domain	32.01	32.14	39
	domain	28.81	29.29	59

[Administrative] Master's Thesis

Administration of Final Theses

CIT Portal

Since **15 January 2024**, all final theses in the TUM School of Computation, Information and Technology (CIT) are managed via the **CIT portal**.

Once you have found a topic and a supervising chair for your thesis, **you will be registered by the supervising chair**. You will receive an e-mail asking you to confirm your thesis registration. Only after you have **confirmed your registration,** the Academic Programs Office will be able to check the admission requirements, and you will receive an email confirming your binding registration for your thesis.

Link